# Introduction

## Motivation I

Active Inference has matured into a broad research program spanning the free energy principle (Friston 2010), discrete state-space formulations of perception and action (Da Costa et al. 2020), and a growing body of pedagogy and reference implementations (Parr et al. 2022; Smith et al. 2022). Recent graphical-specification and message-passing work makes the same practical pressure visible: active-inference agents need representations that can be inspected as model structure and then carried into executable inference updates without being reauthored by hand (Koudahl et al. 2023; Laar et al. 2023; Bagaev and Vries 2021). As the field has grown, so has a quieter problem: the models themselves are difficult to share, reproduce, and compare. A generative model that lives only as a tangle of matrix definitions inside one author's script, a diagram in a slide deck, and a paragraph of prose in a paper has no

single authoritative form. The same model is described three times, in three incompatible media, with no guarantee that they agree. When a reader wants to re-run that model in a different toolkit, they must reverse-engineer it from whichever fragment they happen to have.

This is a reproducibility and interoperability crisis specific to the structure of Active Inference work. The discipline depends on precisely specified state spaces, observation and transition tensors, prior preferences, and policy structures; small notational ambiguities propagate into materially different behaviour. The heterogeneity of the models themselves compounds the problem: the field works in several model kinds at once — discrete categorical POMDPs over finite state spaces, continuous linear-Gaussian state-space models, multi-agent systems whose agents each carry their own generative vocabulary, hierarchical compositions layered over timescales — and each kind

attracts its own notation, its own notebooks, and its own simulation stack. Yet the community has lacked a notation that is simultaneously human-readable, machine-parseable, and faithful to the underlying mathematics across those kinds. Generalized Notation Notation (GNN) was introduced to fill exactly that gap (Smékal and Friedman 2023): a standard, text-based way to write down an Active Inference generative model once, such that every downstream use derives from the same source of truth.

## The GNN Approach I

GNN treats the model specification as a first-class artifact. A model is written in a plain-text language whose syntax captures the components an Active Inference modeller actually reasons about — state factors, observation modalities, the matrices that link them, control structure, and the temporal organization of the model — and whose declared matrix vocabulary fixes which kind of generative model the file denotes: categorical `A`/`B`/`C`/`D`[/`E`] tensors for a discrete state-space model, linear-Gaussian system matrices for a continuous one, per-agent and per-level compositions for structured variants. Because the specification is text, it lives comfortably in version control, diffs cleanly across revisions, and can be authored and reviewed by humans without specialized tooling.

## The GNN Approach I

The defining commitment of GNN is what the project calls the Triple Play: a single text specification is the common origin for three coordinated renderings of the same model. The text form is the authoritative, editable description. From it, GNN produces graphical visualizations that expose the model's factor and dependency structure for inspection and communication. And from the same source it produces executable cognitive models that can actually be run, so that the diagram, the equations, and the running code are all generated from one parsed document rather than three artifacts that merely claim to describe the same model. The Triple Play turns a model from a description scattered across media into one specification with traceable projections (see fig. 3). Shared source identity supports their comparison; parameter, orientation, model-kind, and numerical checks are still required to establish fidelity.

## Contributions I

This work contributes a standard notation together with the infrastructure that makes it usable in practice:

## Contributions I

- **A parseable, human-readable notation** for Active Inference generative models, whose text form is authoritative and from which every other representation is derived.
- **A model-kind taxonomy with per-kind rendering and execution semantics** — discrete categorical, continuous linear-Gaussian, multi-agent, hierarchical, learning, and structural kinds, classified structurally from the declared notation and rendered or recorded-unsupported per backend — so that what a specification can do on a given framework is a checkable property of the model's kind, not a trial-and-error discovery.
- **A 25-step processing pipeline** (0–24), whose stages are implemented across 44 source packages, that carries a specification from parsing and validation through visualization, rendering, and execution.
- **12 rendering backends** (PyMDP, RxInfer.jl, ActiveInference.jl, JAX, DisCoPy, PyTorch, NumPyro, Stan, bnlearn, cpomdp, THRML, ngc-learn) that materialize a single GNN specification as backend-specific model code, of which 12 (PyMDP, RxInfer.jl, ActiveInference.jl, JAX, DisCoPy, PyTorch, NumPyro, Stan, bnlearn, cpomdp, THRML, ngc-learn) have Step 12 execution adapters. Actual execution requires an admitted model, a ready runtime, and a source-bound run receipt.

## Contributions II

- **164 Model Context Protocol tools** that expose the pipeline's capabilities to agentic and programmatic clients, so the notation and its tooling are directly accessible to automated workflows.
- **9-family reliability gates** that exercise the pipeline against a curated set of model families (basics, discrete, continuous, hierarchical, multiagent, precision, structured, gridworld, scaling-study), turning interoperability claims into checks that must pass rather than assertions that are merely made.

The remainder of this manuscript is organized to move from language to mechanism to evidence. sec. 3 states the notation and the model kinds it denotes — the discrete categorical factorization, its four matrices, and its two free-energy objectives as explicit equations (eq. 1 through eq. 8), the continuous linear-Gaussian state-space kind with its optional closed-loop control structure (eq. 9 through eq. 11), and the multi-agent, hierarchical, learning, and structural compositions — and it presents the model-kind table (tbl. 1), the pipeline structure (fig. 2), the Triple Play (fig. 3), and the per-step responsibility table (tbl. 2). sec. 4 follows a specification through the pipeline — parsing, kind-scoped type checking, rendering, execution, and reporting — and presents the backend registry (tbl. 3), the per-kind framework capability matrix (tbl. 4), the

backend registry surface (fig. 4), and the long-running orchestration contracts (fig. 5). sec. 5 reports what the project has produced and how each number is grounded, with the model-family registry (tbl. 5), the family-by-framework coverage (fig. 6), and repository-scale metrics (fig. 7). fig. 1 condenses this same arc — specification, validation, rendering, execution, analysis, and interchange — into the summary panel at the head of the manuscript. Claims of independent re-execution are addressed in sec. 6, the boundaries of the current system in sec. 7, the symbol and construct tables in sec. 9, and full source details in sec. 10. Read in this order, the manuscript moves from why a standard notation is needed, through how GNN realizes it across model kinds, to the scoped evidence available for the notation and its implementations.